1. Gemini Argon 4 (High) and Qwen 3.6-35B-A3B
2. Gemini via https://gemini.google.com/app (with Google AI Pro subscription); Qwen via LM Studio
3.

Generate C++ code using objects, not functions, to implement
the following requirements:
- Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
- Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
o Each digit from 1 to 9 must appear exactly once in each row.
o Each digit from 1 to 9 must appear exactly once in each column.
o Each digit from 1 to 9 must appear exactly once in each block.
- Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
- The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
- It is possible to solve a Sudoku puzzle entirely through logic. For example, in the given puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.
- That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
- Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first

candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).

- This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
- You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
- Write a recursive object-oriented program to find the solution to the five puzzle files.
- The output for each solution, should include:
o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
- The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
- To help you debug your program, solution1.txt is a solution to puzzle1.txt.

puzzle1.txt:

```
5 _ _ _ _ _ 1 7 _
1 _ 6 5 _ 9 _ 4 _
4 7 2 1 _ 6 _ _ _
9 _ _ _ _ _ 5 _ _
_ 1 8 _ 9 5 4 _ _
6 _ _ 4 _ 2 3 8 9
_ 4 _ _ _ _ 9 3 _
_ 9 _ 7 _ 3 _ 5 _
2 6 3 9 5 8 7 1 4
```

solution1.txt:

```
5 3 9 8 2 4 1 7 6
```

1 8 6 5 7 9 2 4 3
4 7 2 1 3 6 8 9 5
9 2 4 3 8 7 5 6 1
3 1 8 6 9 5 4 2 7
6 5 7 4 1 2 3 8 9
7 4 5 2 6 1 9 3 8
8 9 1 7 4 3 6 5 2
2 6 3 9 5 8 7 1 4
Solutions found: 1

4.

a. Gemini

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

// Represents the 9x9 Sudoku grid and handles cell state and rule validation
class SudokuBoard {
private:
    static const int SIZE = 9;
    static const int BLOCK_SIZE = 3;
    char grid[SIZE][SIZE];

    bool isInRow(int row, char digit) const {
        for (int col = 0; col < SIZE; ++col) {
            if (grid[row][col] == digit) return true;
        }
        return false;
    }

    bool isInCol(int col, char digit) const {
        for (int row = 0; row < SIZE; ++row) {
            if (grid[row][col] == digit) return true;
        }
        return false;
    }

    bool isInBlock(int startRow, int startCol, char digit) const {
        for (int r = 0; r < BLOCK_SIZE; ++r) {
            for (int c = 0; c < BLOCK_SIZE; ++c) {
                if (grid[startRow + r][startCol + c] == digit) return true;
            }
        }
        return false;
    }

public:
    SudokuBoard() {
        clear();
    }

    void clear() {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                grid[r][c] = '_';
            }
        }
    }

    bool loadFromFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (!(file >> grid[r][c])) {
                    return false;
                }
            }
        }
        return true;
    }

    bool isValidPlacement(int row, int col, char digit) const {
        int blockStartRow = (row / BLOCK_SIZE) * BLOCK_SIZE;
        int blockStartCol = (col / BLOCK_SIZE) * BLOCK_SIZE;
        return !isInRow(row, digit) &&
               !isInCol(col, digit) &&
               !isInBlock(blockStartRow, blockStartCol, digit);
    }

    // Verifies that the initial pre-filled board doesn't violate Sudoku rules
    bool isInitialBoardValid() const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
```

```
                    const_cast<SudokuBoard*>(this)->grid[r][c] = val;
                    if (!valid) return false;
                }
            }
        }
        return true;
    }

    // Finds the next empty cell ('_'); returns false if all 81 cells are filled
    bool findEmptyCell(int& outRow, int& outCol) const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (grid[r][c] == '_') {
                    outRow = r;
                    outCol = c;
                    return true;
                }
            }
        }
        return false;
    }

    // Deduces the list of valid candidate digits (1-9) for a given cell
    std::vector<char> getCandidates(int row, int col) const {
        std::vector<char> candidates;
        for (char digit = '1'; digit <= '9'; ++digit) {
            if (isValidPlacement(row, col, digit)) {
                candidates.push_back(digit);
            }
        }
        return candidates;
    }

    void setCell(int row, int col, char val) {
        grid[row][col] = val;
    }

    void print() const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                std::cout << grid[r][c] << (c < SIZE - 1 ? " " : "");
            }
            std::cout << "\n";
        }
    }
};

// Encapsulates the depth-first search with backtracking algorithm
class SudokuSolver {
private:
    SudokuBoard board;
    int solutionsCount;

    // Recursive depth-first search with backtracking
    bool solveRecursive() {
        int row = 0, col = 0;

        // Base case: k == 81 cells filled (no empty cells remain)
        if (!board.findEmptyCell(row, col)) {
            ++solutionsCount;
            std::cout << "\nSolution " << solutionsCount << ":\n";
            board.print();
            return true;
        }

        // Deduce valid candidates for the current empty cell
        std::vector<char> candidates = board.getCandidates(row, col);

        // If no candidates exist, we hit a dead end and backtrack
        if (candidates.empty()) {
            return false;
```

```
        for (char candidate : candidates) {
            board.setCell(row, col, candidate);

            // Recursive call with k + 1 cells filled
            if (solveRecursive()) {
                foundSolutionInBranch = true;
            }

            // Backtrack: undo the cell assignment to try the next candidate
            board.setCell(row, col, '_');
        }

        return foundSolutionInBranch;
    }

public:
    SudokuSolver() : solutionsCount(0) {}

    void processPuzzleFile(const std::string& filename) {
        solutionsCount = 0;
        std::cout << "==================================\n";
        std::cout << filename << ":\n";

        if (!board.loadFromFile(filename)) {
            std::cout << "Error: Unable to open or read " << filename << "\n\n";
            return;
        }

        // Print the initial puzzle from the file
        board.print();

        // Check if initial board configuration is legal before searching
        if (!board.isInitialBoardValid() || !solveRecursive()) {
            std::cout << "\nNo solution found\n";
        } else {
            std::cout << "\nSolutions found: " << solutionsCount << "\n";
        }
        std::cout << "\n";
    }
};

// Orchestrates solving the batch of five puzzle files
class SudokuApplication {
private:
    std::vector<std::string> puzzleFiles;
    SudokuSolver solver;

public:
    SudokuApplication() {
        puzzleFiles = {
            "puzzle1.txt",
            "puzzle2.txt",
            "puzzle3.txt",
            "puzzle4.txt",
            "puzzle5.txt"
        };
    }

    void run() {
        for (const std::string& filename : puzzleFiles) {
            solver.processPuzzleFile(filename);
        }
    }
};

int main() {
    SudokuApplication app;
```

b. Qwen

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <iomanip>

using namespace std;

// Forward declaration
class SudokuSolver;

/**
 * Class representing the Sudoku grid and its data.
 */
class SudokuBoard {
private:
    int grid[9][9]; // 0 represents an empty cell
    string filename;
    int solutionsFoundCount;

public:
    SudokuBoard() : solutionsFoundCount(0) {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                grid[i][j] = 0;
            }
        }
    }

    // Load puzzle from file
    bool loadFromFile(const string& fname) {
        filename = fname;
        ifstream file(fname);
        if (!file.is_open()) {
            cerr << "Error: Could not open file " << fname << endl;
            return false;
        }

        int r = 0, c = 0;
        string token;

        // Read tokens (separated by whitespace)
        while (r < 9 && file >> token) {
            if (token == "_") {
                grid[r][c] = 0;
            } else {
                try {
                    grid[r][c] = stoi(token);
                } catch (...) {
                    cerr << "Error parsing number in file." << endl;
                    return false;
                }
            }

            c++;
            if (c == 9) {
                c = 0;
                r++;
            }
        }

        file.close();
        return true;
    }

    // Print the current state of the board
    void printGrid() const {
        for (int i = 0; i < 9; ++i) {
            if (i == 3 || i == 6) cout << "+-------+-------+-------+" << endl;
            else if (i == 0) cout << "|-------+-------+-------|" << endl; // Top border

            for (int j = 0; j < 9; ++j) {
                if (j == 3 || j == 6) cout << "| ";

                if (grid[i][j] != 0) {
                    cout << grid[i][j];
```

```
            cout << "|" << endl;
        }
    }

    // Getters and Setters for the solver to use
    int getCell(int r, int c) const { return grid[r][c]; }
    void setCell(int r, int c, int val) { grid[r][c] = val; }

    string getFilename() const { return filename; }
    int getSolutionsCount() const { return solutionsFoundCount; }
    void incrementSolutionsCount() { solutionsFoundCount++; }

    // Check if placing num at (r, c) is valid according to Sudoku rules
    bool isValid(int r, int c, int num) const {
        // Check Row
        for (int x = 0; x < 9; ++x) {
            if (grid[r][x] == num) return false;
        }

        // Check Column
        for (int x = 0; x < 9; ++x) {
            if (grid[x][c] == num) return false;
        }

        // Check 3x3 Box
        int startRow = r - r % 3;
        int startCol = c - c % 3;
        for (int i = 0; i < 3; ++i) {
            for (int j = 0; j < 3; ++j) {
                if (grid[i + startRow][j + startCol] == num) return false;
            }
        }

        return true;
    }
};

/**
 * Class responsible for solving the SudokuBoard using backtracking.
 */
class SudokuSolver {
private:
    SudokuBoard& board; // Reference to the board being solved

public:
    SudokuSolver(SudokuBoard& b) : board(b) {}

    /**
     * Entry point for the solver. Finds all solutions.
     */
    void solve() {
        findSolution(0, 0);
    }

private:
    /**
     * Recursive helper function to fill cells.
     * Returns true if a solution was found (and printed), false otherwise.
     * Note: To print ALL solutions, we don't return immediately upon finding one;
     * we continue the search after printing.
     */
    bool findSolution(int r, int c) {
        // Base Case: If we have passed the last row, we found a valid solution
        if (r == 9) {
            cout << "Solution:" << endl;
            board.printGrid();
            cout << endl;
            return true;
        }

        // Calculate next cell coordinates
        int nextR = r;
        int nextC = c + 1;
```

```
158         }
159
160         // If current cell is already filled, move to next
161         if (board.getCell(r, c) != 0) {
162             return findSolution(nextR, nextC);
163         }
164
165         // Try digits 1-9 in the empty cell
166         bool foundAny = false;
167         for (int num = 1; num <= 9; ++num) {
168             if (board.isValid(r, c, num)) {
169                 board.setCell(r, c, num); // Make a move
170
171                 // Recurse to next cell
172                 if (findSolution(nextR, nextC)) {
173                     foundAny = true;
174                     board.incrementSolutionsCount();
175                     // We do NOT return false here. We want to find ALL solutions.
176                     // However, we must backtrack after printing so the loop can try the next number.
177                 }
178
179                 // Backtrack: Remove the digit to try the next possibility
180                 board.setCell(r, c, 0);
181             }
182         }
183
184         return foundAny;
185     }
186 };
187
188 int main() {
189     // Loop through puzzle files 1 to 5
190     for (int i = 1; i <= 5; ++i) {
191         string filename = "puzzle" + to_string(i) + ".txt";
192
193         cout << "Processing: " << filename << endl;
194         cout << "Original Puzzle:" << endl;
195
196         SudokuBoard board;
197         if (board.loadFromFile(filename)) {
198             board.printGrid();
199
200             // Create solver object and solve
201             SudokuSolver solver(board);
202             solver.solve();
203
204             cout << "Solutions found: " << board.getSolutionsCount() << endl;
205         } else {
206             cout << "Failed to load puzzle." << endl;
207         }
208
209         cout << "----------------------------------------" << endl;
210     }
211
212     return 0;
213 }
```

5.

a. Gemini

i. Correctness: Handles the happy path appropriately. When I corrupt a puzzle (such as by removing an underscore), the

program appropriately says it cannot read the file, although this should instead indicate the file's formatting is bad.

ii. Execution time: The program executes within the desired timeframe, but can still be optimized. SudokuBoard.findEmptyCell can be optimized to start searching at a given space, rather than processing the entire grid all over again. The vector in SudokuSolver.solveRecursive could be replaced with a static array of size 9 to avoid allocating on the heap for every recursive call. The same thing applies to SudokuBoard.getCandidates. Also for getCandidates, each digit is checked over the entire board every single time, instead of storing what digits are present in each row and column.

iii. Space Complexity: The grid is stored as an array of characters, versus being stored as a type like uint16, because there are only 10 possible values (0-9 and _).

iv. Maintainability: Responsibilities are separated out into each class appropriately. Static constants are used to avoid magic numbers. SudokuBoard.isInitialBoardValid is a const function, but because of const_cast, it is modifying data, which a const function should not be doing.

b. Qwen

i. Correctness: Handles the happy path, but miscounts how many solutions were found. When I make a puzzle invalid (such as by having two numbers in the same row), it just says there's 0 solutions. When I corrupt a puzzle by removing a character, it reads each row into the next row, and then fills in with underscores at the end, instead of recognizing it as invalid. Some functionality is also relegated to the main function, instead of an object. The output uses special formatting, which deviates from the desired output.

ii. Execution time: The program executes within the desired timeframe. SudokuBoard.isValid checks over each digit of the entire board, rather than storing what digits are present in each

row and column. SudokuSolver.findSolution is executed on every cell, not just empty cells.

iii. Space Complexity: The grid is stored as whole 4 byte integers, when only 10 possible values exist per cell.

iv. Maintainability: solutionsFoundCount is stored in the SudokuBoard itself, which makes it take some responsibility away from the solver. Uses a useless forward declaration. vector and iomanip are included, but never used. using namespace std means that namespace collisions could occur.

c. I choose Gemini's code because it is nearly entirely correct in its behavior and has issues that can be easily resolved.

d.

i. Correctness: Improved input validation by distinguishing file-opening errors from formatting errors. The program now checks for invalid characters, missing or extra cell data, and duplicate clues in the initial Sudoku board. Malformed input produces a more appropriate error message.

ii. Execution time: Optimized the solver by allowing findEmptyCell() to resume searching from a specified position rather than starting from the beginning each time. Replaced the candidate vector with a fixed-size array to avoid repeated heap allocations during recursive solving. Also added row, column, and block lookup tables to make candidate validation faster.

iii. Space Complexity: Changed the grid from character storage to uint16_t numeric values, with 0 representing an empty cell. Also replaced the candidate vector with a fixed-size array.

iv. Maintainability: Removed the need for const_cast in isInitialBoardValid() by using local arrays for validation, ensuring the method does not modify the board.